Cedar’s dot-notation and bracket syntax work consistently. Dot notation is cleaner, but brackets can be helpful when attribute names include special characters or need to be computed. In short, Cedar offers predictable, straightforward access to deeply nested data, regardless of slight differences in how the data is modeled or represented in the entity store or incoming request.

### 7.3.3 Combining results from multiple policies

Once Cedar has filtered and evaluated individual policies, it needs to combine their results into a single decision. It does this using a simple, but powerful, deny-overrides model:

1. If any applicable policy is `forbid` → `Deny`. If even one policy with effect `forbid` evaluates to true, Cedar immediately returns `Deny`, regardless of any matching permit policies. This ensures that explicit denials always take precedence.
2. If any applicable policy is a `permit` policy → `Allow`. If there are no matching `forbids` but at least one `permit` policy evaluates to `true`, Cedar returns `Allow`.
3. Otherwise → `Deny` by default. If no policies apply or any applicable `permit` policies evaluate to `false`, Cedar returns `Deny`. This deny-by-default approach makes explicit permissions the only way to grant access.

This model offers important advantages:

- **Safety-first defaults.** Access is granted only when explicitly allowed, reducing exposure from missing or incomplete policies.
- **Policy guardrails.** Universal deny rules like “no non-managed devices” can always block access, even if other policies allow it.
- **Order independence.** Since denials hold the ultimate authority, there's no need to worry about policy order or priority conflicts. Cedar’s logic stays straightforward and predictable.

Cedar’s decision model can be summarized by these three rules: